

```

import time

class FakeStripeAPI:
    def __init__(self):
        self.charges = []
    def charge(self, amount, idempotency_key=None):
        # Flawed: No idempotency key used
        print(f"[STRIPE] Charging ${amount}...")
        self.charges.append(amount)
        return {"status": "success"}

class FakeDatabase:
    def __init__(self):
        self.processed_events = set()
    def commit(self):
        print("[DB] Transaction committed.")
    def rollback(self):
        print("[DB] Transaction rolled back.")

class FlawedPaymentService:
    def __init__(self, db, stripe):
        self.db = db
        self.stripe = stripe
    def process_payment(self, command_id, amount):
        print(f"\n--- Processing {command_id} ---")
        if command_id in self.db.processed_events:
            print("[DB] Duplicate command found. Skipping.")
            return

        # Simulate local DB transaction
        self.db.processed_events.add(command_id)
        self.stripe.charge(amount) # External side effect

        # Simulate crash AFTER API call but BEFORE commit
        raise Exception("CRASH: DB connection dropped before commit!")

if __name__ == "__main__":
    db = FakeDatabase()
    stripe = FakeStripeAPI()
    service = FlawedPaymentService(db, stripe)

    # Kafka delivery 1
    try:
        service.process_payment("CMD-PAY-101", 10.00)

```

```
except Exception as e:
```

```
    print(e)
```

```
    db.rollback()
```

```
# Kafka redelivery (at-least-once)
```

```
try:
```

```
    service.process_payment("CMD-PAY-101", 10.00)
```

```
except Exception as e:
```

```
    print(e)
```

```
    db.rollback()
```

```
print(f"\nTotal charges: ${sum(stripe.charges)}")
```